

Martin Fowler

003-1040559

1250 003-77156.8

1760 0009-14563.7

73273

Martin Fowler

Ward Cunningham

Andrew Hunt

Jon Kern

Kent Beck

Ron Jeffries

Brian Marick

Mike Beedle

Ken Schwaber

Dave Thomas

Robert C. Martin

Jeff Sutherland

Arie van Bennekum

James Grenning

Jim Highsmith

Steve Mellor

Alistair Cockburn

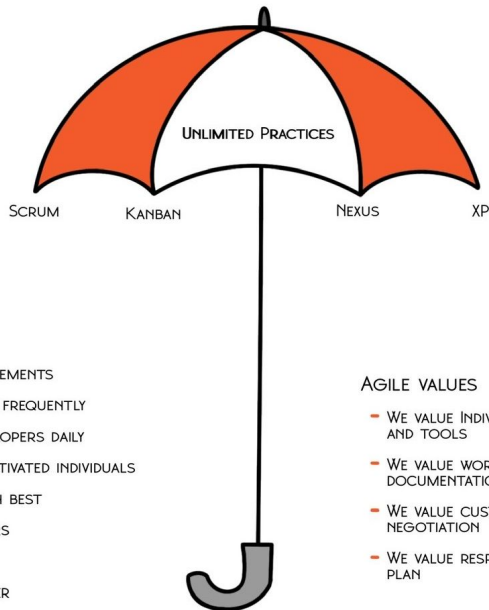
003-1040559

1250 003-77156.8

1760 0009-14563.7

73273

AGILE IS AN UMBRELLA TERM FOR VALUE DELIVERY FRAMEWORKS BASED ON THE AGILE VALUES AND PRINCIPLES IN THE AGILE MANIFESTO.



12 PRINCIPLES

- SATISFY THE CUSTOMER
- WELCOME CHANGING REQUIREMENTS
- DELIVER WORKING SOFTWARE FREQUENTLY
- BUSINESS WORKS WITH DEVELOPERS DAILY
- BUILD PROJECTS AROUND MOTIVATED INDIVIDUALS
- FACE-TO-FACE CONVERSATION BEST
- WORKING SOFTWARE MATTERS
- CONSTANT PACE
- HIGH QUALITY CODE IS FASTER
- MAXIMIZE WORK NOT DONE
- SELF-ORGANIZING TEAMS DELIVER QUALITY
- FEEDBACK LOOPS USED TO IMPROVE

AGILE VALUES

- WE VALUE INDIVIDUALS AND INTERACTIONS OVER PROCESSES AND TOOLS
- WE VALUE WORKING SOFTWARE OVER COMPREHENSIVE DOCUMENTATION
- WE VALUE CUSTOMER COLLABORATION OVER CONTRACT NEGOTIATION
- WE VALUE RESPONDING TO CHANGE OVER FOLLOWING A PLAN

ONLY TO REMEMBER WHAT IS AGILE MANIFESTO



Martin Fowler About Himself





I've become a big fan of agile approaches and the resulting focus on evolutionary software architecture. I don't come up with original ideas, but do a pretty good job of recognizing and packaging the ideas of others, or as Brian Foote describes me: "an intellectual jackal with good taste in carrion".



01

02

03

04

05

06

01

02

03

04

05

06

CHRONOLOGY OF MARTIN FOWLER BOOKS

1994



Analysis Patterns

Reusable
Object Models

2000



**Planning
Extreme**

Programming

by Kent Beck
and Martin
Fowler

2002



**Patterns of
Enterprise**

by Kent Beck
and Martin
Fowler

2003



UML

UML Distilled



CHRONOLOGY OF MARTIN FOWLER BOOKS

2009



Refactoring

Ruby Edition

2010



**Domain Spec
Languages**
(DSLs)

2012



NoSQL Distilled

A Brief Guide
to the
Emerging
World of
Polyglot
Persistence

2018



Refactoring

transform code
with safe and
rapid process



PAIR PROGRAMMING



Ward Cunningham

Kent Beck

Mike Beedle

Martin Fowler

Dave Thomas

Arie van Bennekum

James Grenning

Alistair Cockburn

Andrew



Gang Of Four

The authors of the [DesignPatternsBook](#) came to be known as the "Gang of Four." The name of the book ("Design Patterns: Elements of Reusable Object-Oriented Software") is too long for e-mail, so "book by the gang of four" became a shorthand name for it. After all, it isn't the ONLY book on patterns. That got shortened to "GOF book", which is pretty cryptic the first time you hear it.

- [ErichGamma](#)
- [RichardHelm](#)
- [RalphJohnson](#)
- [JohnVlissides](#)

"Gang of Four" was originally a reference to certain people of note in Maoist China. Both the band and the authors of the [DesignPatternsBook](#) are named after them. See http://news.bbc.co.uk/1/hi/english/static/special_report/1999/09/99/china_50/gang.htm

Sadly, [JohnVlissides](#) passed away on November 24 2005 after a long illness. His home page [JohnVlissides](#) has many many great stories about his life from his friends and colleagues.

[GOF:PageNo] is an increasingly common acronym for the patterns found in this book. i.e

- [GOF:233] - Command Pattern.
- [GOF:163] - Composite Pattern.

Cool ! I would suggest using locale-enabled notation to provide this kind of information to non-English readers:

- [GOF{fr}:271] - Pattern Commande.
- [GOF{fr}:189] - Pattern Composite.

-- [GuillaumeBertrand](#)

>C'est nickel!< Only ... and, not having seen the book, I hesitate here ... are pages not confounding? Could not pattern number serve better in the form [GOF{gr}:162- Patter Whatever]? -- [BenTremblay](#)



Word C

Kent Beck

Mike Beedle

Dave Thomas

Arie van Bennekum

Alistair Cockb



- Contents
- Foreword
- Preface
- Acknowledgments
- About the Authors
- Chapter 1: Refactoring, a First Exam
- Chapter 2: Principles in Refactoring
- Chapter 3: Bad Smells in Code
 - Duplicated Code
 - Long Method
 - Large Class
 - Long Parameter List
 - Divergent Change
 - Shotgun Surgery
 - Feature Envy
 - Data Clumps
 - Primitive Obsession
 - Case Statements
 - Parallel Inheritance Hierarchies
 - Lazy Class
 - Speculative Generality
 - Temporary Field
 - Message Chains
 - Middle Man
 - Inappropriate Intimacy
 - Alternative Classes with Differen
 - Incomplete Library Class
 - Data Class
 - Refused Bequest
 - Comments
 - Metaprogramming Madness
 - Disjointed API
 - Repetitive Boilerplate
- Chapter 4: Building Tests
- Chapter 5: Toward a Catalog of Ref
- Chapter 6: Composing Methods
- Chapter 7: Moving Features Between
- Chapter 8: Organizing Data
- Chapter 9: Simplifying Conditional
- Chapter 10: Making Method Calls Si
- Chapter 11: Dealing with Generaliza
- Chapter 12: Big Refactorings
- Chapter 13: Putting It All Together

Chapter 3

Bad Smells in Code

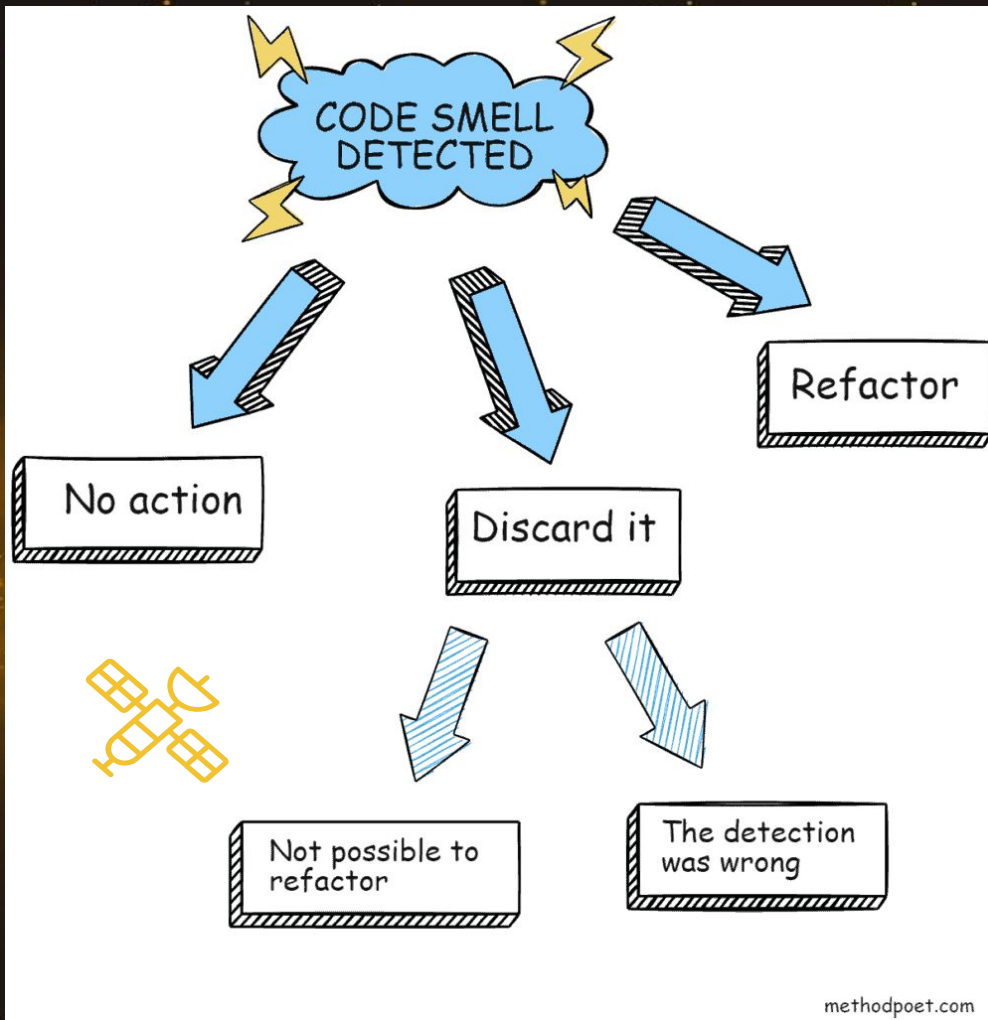


If it stinks, change it.

Grandma Beck, discussing child-rearing philosophy

By now you have a good idea of how refactoring know how doesn't mean you know when. Deciding and when to stop, is just as important to refactoring the mechanics of a refactoring.

Now comes the dilemma. It is easy to explain how



Refactoring is a disciplined technique for restructuring an existing body of code, altering its internal structure without changing its external behavior.



“Any fool can write code that a computer can understand. Good programmers write code that humans can understand.”

Martin Fowler



01

02

03

04

05

06

01

02

03

04

05

06



“Any fool can write code that a computer can understand. Good programmers write code that humans can understand.”

Martin Fowler



01

02

03

04

05

06

01

02

03

04

05

06



“I’m not a great programmer; I’m just a
good programmer with great habits.”

Martin Fowler



01

02

03

04

05

06

01

02

03

04

05

06



“Whenever I have to think to understand what the code is doing, I ask myself if I can refactor the code to make that understanding more immediately apparent.”

Martin Fowler



01

02

03

04

05

06

01

02

03

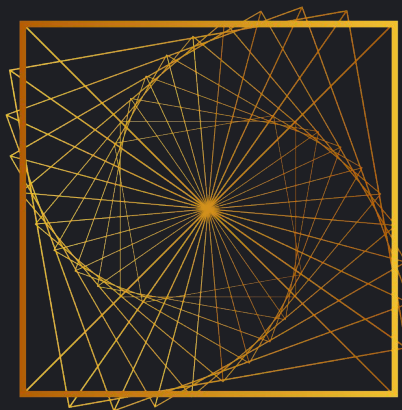
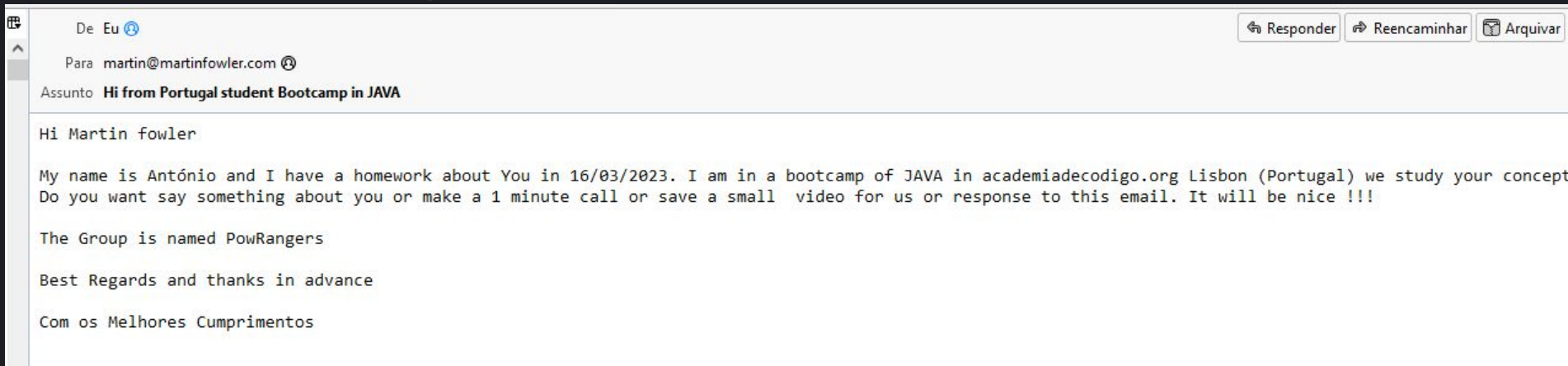
04

05

06

“the no is always guaranteed”

Try



Catch

I “received” a short call audio (Facetime iPad) from Martin explaining that he could not write or film any video due to copyrights. But he properly pronounced the word **POW** and wished everyone the best for all.

003-1040559 1250 003-77156.8 1760 0009-14563.7 73273

Martin Fowler

Refactoring

JUnit

UML

SQL

NoSQL

MicroServices

Design Patterns

Software Engineering

OOP

Testing

THANKS !!!



003-1040559

1250 003-77156.8

1760 0009-14563.7

73273